Design and Bus-Level Verification of a RISC-V-Compatible Memory-Mapped CRC-32 Accelerator Using AXI-Lite

Rahul Nerella, Uttiyo Majumder, and Kevin Velanjickal
Department of Electronics and Communication Engineering
Vellore Institute of Technology, Vellore, India
V-SPACE FPGA Build Challenge 2026

***Abstract*—This paper presents the design and simulation-level verification of a memory-mapped CRC-32 accelerator connected through a single-master AXI4-Lite fabric to a 64-KiB byte-enabled RAM. A bridge translates a compact RISC-V-style memory-port contract (valid, ready, address, write data, and byte strobes) into AXI-Lite transactions. The peripheral implements the reflected CRC-32 polynomial `0xEDB88320`, an initial state of `0xFFFFFFFF`, strobed-byte processing, and final inversion on result reads. Two archived ModelSim testbenches report successful completion: a bus-functional-model (BFM) suite covering RAM integrity, byte strobes, transaction sequencing, backpressure, decode errors, CRC golden-vector verification, and reset recovery; and a SoC-boundary suite that checks RAM round trips, partial writes, CRC-32 for ASCII "123456789," and internal output-port behavior. Observed values include `0xCAFEBABE` for a RAM read-back, `0xCAFEAABE` after a byte-lane update, and the standard CRC-32 result `0xCBF43926`. The current archive does not include or instantiate the PicoRV32 core in the tested configuration; the testbench drives the memory-port interface directly. No synthesis report, timing-closure report, or physical FPGA demonstration is included, so the findings support bus/peripheral functional verification rather than complete CPU-in-loop or hardware validation.**

***Index Terms*—RISC-V, CRC-32, FPGA, AXI4-Lite, memory-mapped accelerator, bus interconnect, byte strobes, SystemVerilog verification.**

I. Introduction

Cyclic redundancy checks (CRCs) are widely used to detect accidental data corruption in communication and storage systems. Although a CRC can be evaluated in software, a hardware peripheral can offload repeated polynomial updates from a processor and provide a reusable example of memory-mapped accelerator integration. Such an integration is only reliable when transaction handshakes, address decoding, byte enables, completion responses, and error paths are all verified.

Experiment 2 builds a bridge from a compact RISC-V-style memory interface to an AXI-Lite interconnect, a 64-KiB RAM slave, and a register-mapped CRC-32 peripheral. The project uses explicit address constants in a SystemVerilog package and a single-outstanding policy to simplify transaction ordering. The design is exercised at two levels: direct AXI-Lite bus-functional testing and a boundary-level testbench that drives the same memory-port signals expected by PicoRV32.

The scope must be stated precisely. The current submission archive reserves a directory for external PicoRV32 source but does not contain the core RTL, and the checked-in top-level BFM configuration does not instantiate it. Thus, this paper documents a RISC-V-compatible interface and bus-level accelerator path; it does not claim a completed CPU-in-loop boot or a demonstrated processor executing firmware.

TABLE I
Memory-Mapped Regions in the Submitted RTL

Region/register	Address	Function
RAM	`0x00000000`–`0x0000FFFF`	64-KiB memory; 32-bit words with byte strobes.
CRC_DATA	`0x10000000`	Write strobed bytes into running CRC state.
CRC_CTRL	`0x10000004`	Any write resets the CRC state to all ones.
CRC_RESULT	`0x10000008`	Read final-complemented CRC result.
CRC_STATUS	`0x1000000C`	Bit 0: busy; bit 1: done/not busy.
OUT_PORT	`0x80000000`	Internal write monitor; reads return DECERR.
Unmapped	Other addresses	Complete internally with decode-error response.

II. Architecture and Address Map

A. System-Level Structure

Fig. 1 reproduces the architecture image included in the project ZIP. The leftmost block denotes the memory-port contract that the bridge is designed to support. In the archived simulations, testbench tasks supply that contract directly. The active data path is the memory-port boundary, bridge, interconnect, RAM/CRC target logic, and response path.

B. Memory Map

The package `mem_map_pkg.sv` centralizes address regions and response codes. Table I summarizes the map from the checked-in RTL.

The interconnect decodes writes and reads independently and latches the destination for each outstanding transaction. The RAM and CRC regions are forwarded to their respective slaves. The output port and unmapped space use an internal response path. The design supports one outstanding read and

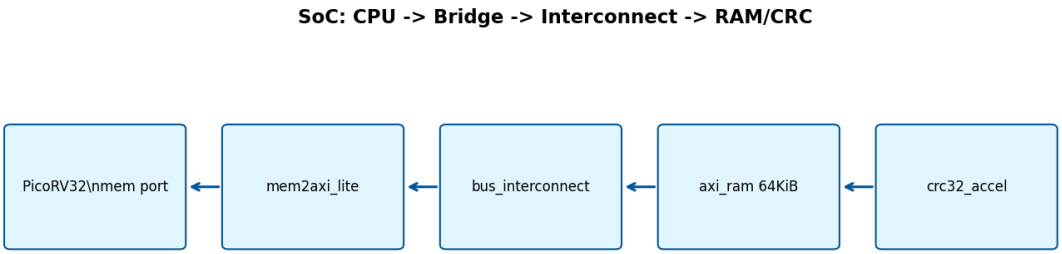


Fig. 1. Project-provided system diagram. The PicoRV32-labelled port is an interface target, not evidence of a PicoRV32 instance in the archived testbench.

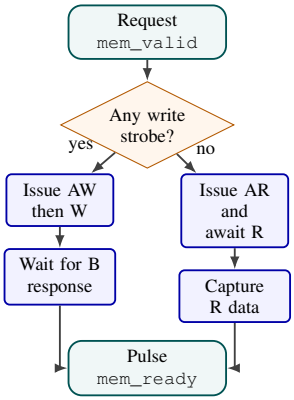


Fig. 2. Simplified request sequencing of the checked-in memory-port bridge. The bridge serializes AW then W and permits only one in-flight transaction.

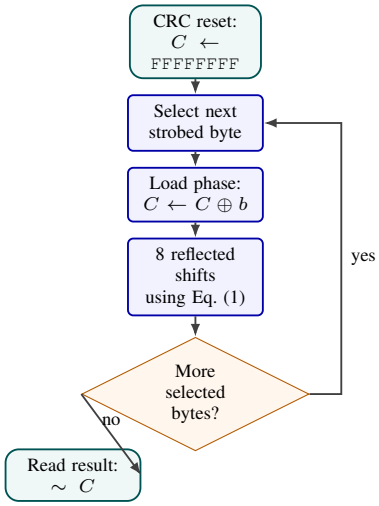


Fig. 3. Conceptual byte-serial CRC data path. Unstrobed byte lanes are skipped; the final inversion is applied only when the result register is read.

one outstanding write at a time; it does not implement general multi-master arbitration or multiple IDs.

C. Memory-Port to AXI-Lite Bridge

The bridge accepts a request when `mem_valid` is asserted and the bridge is armed for a new request. A nonzero write-strobe mask selects a write; a zero mask selects a read. On the write path, AW is issued before W, then the bridge waits for B. On the read path, AR is issued and the returned R data is captured before `mem_ready` is pulsed. The bridge uses a request re-arm flag so a memory master that holds `mem_valid` until completion does not accidentally launch the same operation twice.

III. CRC-32 Peripheral and RTL Design

A. CRC Update Equation

The CRC block implements the reflected CRC-32 update. It starts with $C_0 = \texttt{0xFFFFFFFF}$. For each selected byte b, the byte is XORed into the low eight bits of the running register. Eight reflected shifts then follow. At each shift, the least-significant bit selects whether the reflected polynomial is XORed:

$$C_{i+1} = \begin{cases} (C_i \gg 1) \oplus \texttt{0xEDB88320}, & C_i[0] = 1, \\ C_i \gg 1, & C_i[0] = 0. \end{cases} \tag{1}$$

The output register is complemented when `CRC_RESULT` is read. Complementing only at result-read time is important when a message is accumulated across multiple writes. Byte lanes are processed from byte index 0 through 3, and only lanes selected by `WSTRB` are included. The golden test feeds the bytes of "123456789" in little-endian word order and expects `0xCBF43926`.

B. Module Responsibilities

The primary modules and their roles are summarized in Table II. `axi_ram.sv` infers a word array with per-byte write enables. `bus_interconnect.sv` latches address selections and routes address, data, and response channels. `mem2axi_lite.sv` implements the serialized transaction FSM. `crc32_accel.sv` handles control/data/result/status accesses and the CRC update loop.

IV. Verification Methodology

Verification uses two benches. The bus BFM directly checks the AXI-Lite fabric and its RAM/CRC targets. The top-level bench stimulates the simple memory-port interface and executes a fixed transaction sequence that mirrors the intended

TABLE II
PRIMARY RTL MODULES

Module/file	Responsibility
`mem_map_pkg.sv`	Named regions and AXI response constants.
`mem2axi_lite.sv`	Memory-port request capture and AXI-Lite sequencing.
`bus_interconnect.sv`	Address decode, latched routing, internal DECERR/OUT_PORT replies.
`axi_ram.sv`	16,384 32-bit words; 64-KiB byte-enabled memory.
`crc32_accel.sv`	CRC register FSM, byte strobes, bitwise update, status and result.
`soc_top.sv`	Top-level wiring in BFM build configuration.

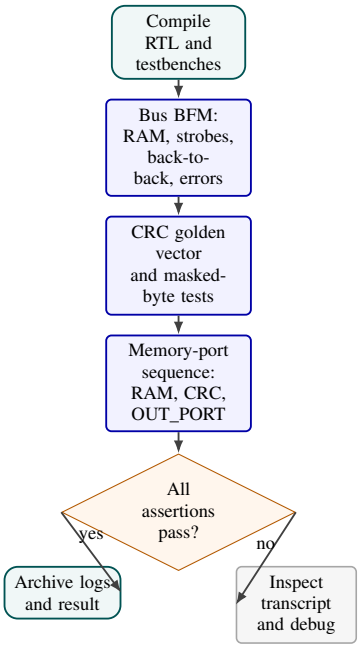


Fig. 4. Verification stages represented by the archived bus and top-level testbenches. The flow is simulation-only and does not include synthesis or board testing.

software ordering. Stimulus is driven away from the active clock edge, and the top-level task waits for `mem_ready` to return low before launching the next operation to avoid consuming a stale completion pulse.

The BFM covers random RAM write/read integrity with selected byte lanes, back-to-back accesses, simultaneous address/data-valid handling, write-channel backpressure, decode-error responses, CRC-32 of "123456789," CRC input byte strobes, and reset recovery. The boundary bench checks the values listed in Table IV. Archived rerun summaries dated 2026-10-10 report zero testbench errors and both benches print their all-checks-passed verdicts.

A. Bus BFM Coverage Matrix

Table III maps the named BFM scenarios in the source comments to the behavior they exercise. The archived final verdict is aggregate: all enabled assertions completed with zero reported errors. The table is not a substitute for a formal protocol-compliance claim.

TABLE III
BUS BFM SCENARIOS AND ARCHIVED VERDICT

ID	Scenario	Verdict
P1	Random RAM read/write integrity and byte-lane merge	No assertion errors
P2	Back-to-back transactions with no idle gap	No assertion errors
P3	Simultaneous AW and W valid handling	No assertion errors
P4	Delayed W-channel ready / backpressure	No assertion errors
P5	Unmapped address DECERR and subsequent bus recovery	No assertion errors
P6	CRC-32 golden vector for "123456789"	Exact value match
P7	CRC byte-strobe inclusion/exclusion	No assertion errors
P8	Reset during a transaction and post-reset recovery	No assertion errors

TABLE IV
TOP-LEVEL MEMORY-PORT TEST RESULTS

Test	Observed result
RAM store/load	Written `0xCAFEBABE`; read-back matched.
Partial byte write	Byte lane 1 changed to `0xAA`; result `0xCAFEAABE`.
CRC-32 golden vector	"123456789" produced `0xCBF43926`.
Output-port write	`OUT_PORT` write acknowledged internally.
Output-port read	Completed without hanging; internal path returned DECERR (data marker `0xBAD0BA5E`).
Bus BFM verdict	`ALL CHECKS PASSED`; archived summary records zero errors.
Boundary bench verdict	`ALL CHECKS PASSED`; archived summary records zero errors.

V. RESULTS

A. Functional Simulation Results

Table IV summarizes the reproducible values in `Simulation/rerun_2026-10-10.log`. The CRC value is the primary arithmetic check; RAM and strobe checks test that the bus-to-slave path preserves data semantics.

Fig. 5 is the experiment summary graphic provided in the ZIP. Fig. 6 is the recorded transaction latency plot generated from `tb_soc_top.sv` timestamp instrumentation. Its title uses the name "PicoRV32 memory-interface" from the project artifact, but the plotted requests were initiated by the testbench, not by an instantiated CPU. The graph spans approximately 4–45 cycles for valid-to-ready and 5–45 cycles for request-to-completion; the long transactions are consistent with a multi-byte bit-serial CRC update. These values are testbench-specific observations, not a timing-closure result or a general performance benchmark.

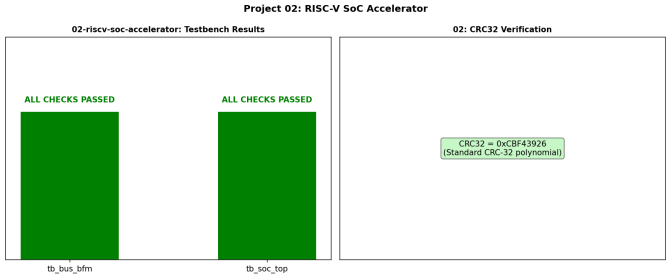


Fig. 5. Project-provided summary graphic showing the archived pass labels and CRC golden value. The primary evidence remains the simulation transcripts and self-checking testbench verdicts.

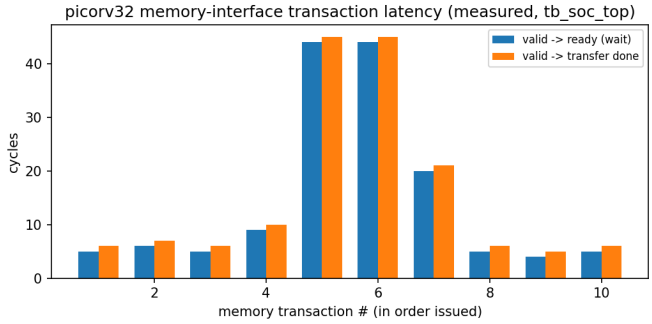


Fig. 6. Recorded boundary-port transaction latencies, measured in clock cycles by the testbench. This is not a CPU-executed measurement.

B. Interpretation

The two passing testbench verdicts indicate that the submitted bridge/fabric/target configuration completed its defined simulation tests without reported assertions. The golden-vector match validates the implemented CRC convention for the tested input sequence; the byte-strobe test additionally checks that unselected lanes do not contribute to the running CRC. The RAM test confirms byte-lane merge behavior for the documented case. These results do not establish complete conformance to every AXI4-Lite timing scenario, throughput under concurrency, or correct operation with a real processor attached.

VI. Hardware Implementation Status and Limitations

The project includes a Nexys A7 constraint template targeting an Artix-7 device. The file is explicitly marked as a template and must be reconciled against the exact board revision and official master XDC before implementation. There is no verified synthesis result, utilization report, static timing report, bitstream, board photograph, or on-board trace in the supplied Experiment 2 artifacts. Hardware operation is therefore pending.

The most important integration limitation is the absent PicoRV32 source and instantiation. The `USE_PICO` parameter is present in `soc_top.sv`, but the tested `SOC_TOP_WITH_BFM` branch wires the bridge and fabric without a CPU core, while the alternative branch reports that no fabric is wired. The firmware ELF is also not present in the package. A complete end-to-end claim requires sourcing and licensing the intended core, integrating it into the top-level, building a memory image, running a core-in-loop regression, and then synthesizing and testing on the selected FPGA board.

Other design limits include a single-master, single-outstanding transaction model and a serialized AW-then-W bridge. The bit-serial CRC performs one load phase and eight shift phases for each selected byte, trading throughput for simple RTL. Future work should include a real core integration, broader protocol assertions, byte/word boundary and malformed-access tests, synthesis/resource reporting, static timing analysis, and physical FPGA validation.

VII. Applications

A memory-mapped CRC peripheral is useful for packet validation, storage integrity checks, firmware self-test, and accelerator-interface education. The bridge and interconnect are also reusable examples of how a compact processor memory port can be adapted to a standard peripheral fabric. The same verification approach—direct BFM tests followed by boundary-level sequencing—can be extended to checksum, cryptographic, or data-movement peripherals.

VIII. Conclusion

This experiment implements and simulates a RISC-V-compatible memory-port bridge, an AXI-Lite decoder/interconnect, byte-enabled 64-KiB RAM, and a memory-mapped CRC-32 peripheral. The archived tests report correct RAM read-back, partial-byte write merging, a CRC-32 result of `0xCBF43926` for "123456789," and clean completion of the BFM and boundary testbenches. The evidence is limited to simulation. The absent CPU integration, unverified board constraints, and lack of synthesis and physical FPGA results remain open tasks before the project can be described as a fully demonstrated PicoRV32-based SoC.

Acknowledgment

The authors acknowledge the V-SPACE FPGA Build Challenge 2026 framework and the maintainers of the cited protocol and device documentation.

References

[1] Arm Ltd., *AMBA AXI and ACE Protocol Specification*, AXI4-Lite protocol documentation. [Online]. Available: https://developer.arm.com/documentation/ihi0022/latest

[2] C. Wolf, "PicoRV32: A Size-Optimized RISC-V CPU," YosysHQ. [Online]. Available: https://github.com/YosysHQ/picorv32. Cited as an integration reference; core RTL is not present in this submission archive.

[3] IEEE, *IEEE Standard for Ethernet*, IEEE Std 802.3, CRC-32 frame-check sequence specification.

[4] Digilent, "Nexys A7 Reference Manual." [Online]. Available: https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual

[5] Experiment source artifacts: `mem_map_pkg.sv`, `mem2axi_lite.sv`, `bus_interconnect.sv`, `axi_ram.sv`, `crc32_accel.sv`, `tb_bus_bfm.sv`, and `tb_soc_top.sv`, V-SPACE FPGA Build Challenge 2026 submission archive.